Multi Container Runtime

Danyl Bince (PES2UG24AM050)
Chiranth D Nandi (PES2UG24AM048)

Linux Namespaces | Pipes & IPC | Kernel Module | CFS Scheduling | POSIX Sync

OS MINI PROJECT

System Architecture

engine.c

User-Space Supervisor

Container lifecycle · CLI commands · Logging pipeline · Signal handling · select() event loop

monitor.ko

Kernel Module

RSS enforcement · Timer callback · softirq context · ioctl interface · spinlock protection

Bounded Buffer

Producer-Consumer Queue

Decouples pipe reads from disk I/O · mutex + cond var · Backpressure via blocking producers

Namespaces

PID + UTS + Mount

Per-container process tree · private hostname · isolated mount table · clone() with flags

All four systems interlock: namespaces isolate, pipes stream, buffer absorbs, kernel enforces.

Linux Namespaces & Isolation

What is a Namespace?

A namespace is a kernel abstraction that makes a process believe it has an exclusive, private view of a system resource.

Without namespaces every process on Linux shares a single flat global view.

This runtime uses three: PID, UTS, and Mount — sufficient to demonstrate meaningful isolation.

Namespace	Clone Flag	What It Isolates
PID	CLONE_NEWPID	Process IDs. Container starts at PID 1
UTS	CLONE_NEWUTS	Hostname and NIS domain name
Mount	CLONE_NEWNS	Filesystem mount table. chroot becomes safe

PID Mapping Example

HOST VIEW	CONTAINER VIEW
PID 1 = systemd	PID 1 = /bin/sh ← same kernel PID
PID 4521 = engine	PID 2 = cpu_hog
PID 4522 = sh (alpha)	(no visibility above PID 1)

clone() — create container process

```
pid_t pid = clone(
  child_fn,        // container entry point
  stack_top,       // x86 stacks grow DOWN
  CLONE_NEWPID |
  CLONE_NEWUTS |
  CLONE_NEWNS  |
  SIGCHLD,         // enables waitpid()
  &cfg
);
```

Pipes & Inter-Process Communication

CHANNEL A — Pipe (log data)

container stdout/stderr → pipe buffer (64 KiB FIFO) → supervisor reader thread

- pipe2(pipefd, O_CLOEXEC)
- dup2(pipefd[1], STDOUT_FILENO) ← redirect stdout to pipe
- close(pipefd[1]) in parent ← enables EOF detection

CHANNEL B — UNIX Socket (CLI control)

/tmp/mini_runtime.sock

- engine start / stop / ps / logs
- Fixed-size packed structs — no framing
- __attribute__((packed)) prevents padding mismatches
- Kernel passes peer PID/UID via SO_PEERCRED

⚠ Critical: close(pipefd[1]) in parent — EOF on a pipe occurs ONLY when ALL write ends are closed. If the parent holds the write end open, the reader thread blocks forever waiting for data that never ends.

Wire Protocol — packed structs over socket

```
typedef struct { cmd_kind_t kind; char id[ID_LEN]; char rootfs[PATH_MAX]; char command[256];
  unsigned long soft_limit_bytes; unsigned long hard_limit_bytes; int nice_value;
} __attribute__((packed)) ctrl_request_t;
```

Bounded Buffer — Producer-Consumer Sync

Ring Buffer

↑ head ↑ tail

Mutex

Binary lock — only one thread modifies head/tail/count at a time. pthread_mutex_lock/unlock.

Condition Variable

Waiting point tied to a predicate. pthread_cond_wait() atomically releases mutex AND sleeps — eliminates lost wakeup.

while() not if()

POSIX permits spurious wakeups. if() lets a woken thread proceed on a false alarm — causes overflow. while() rechecks every time.

The Lost Wakeup Bug — Why It Cannot Happen Here

Thread A checks count == 64 (full) → about to sleep. Thread B decrements count to 63 → signals not_full. Thread A sleeps forever (missed signal). pthread_cond_wait() prevents this: it holds the mutex during predicate check and atomically releases + sleeps. B cannot signal between check and sleep — it must acquire the same mutex.

```
bb_push: pthread_mutex_lock → while(count==CAP) cond_wait(&not_full, &mutex) → insert → count++ → signal(&not_empty) → unlock
```

Kernel Module: monitor.ko

User Space (ring 3): engine.c — protected; can crash safely | Kernel Space (ring 0): monitor.c — no protection; one bad pointer = kernel panic

ioctl — Crossing the Kernel Boundary

```
/* User space */
ioctl(monitor_fd, MONITOR_REGISTER, &req);

/* Kernel — monitor_ioctl() */
struct monitor_register_req req;
if (copy_from_user(&req,
     (void __user *)arg,
     sizeof(req)))
  return -EFAULT;
// NEVER dereference arg directly!
```

Why Enforce in Kernel Space?

1. CPU starvation immunity
Timer fires regardless of user-space scheduling.

2. Compromise resistance
Container cannot ptrace() or kill a kernel timer.

3. Authoritative data
Reads directly from mm_struct — kernel ground truth.

4. Unstoppable SIGKILL
force=1 bypasses signal blocking; delivered next timeslice.

Soft vs Hard Limits

Limit	Threshold	Action
Soft	soft_limit_bytes	Log kernel warning (once per container)
Hard	hard_limit_bytes	send_sig(SIGKILL, task, 1) — force=1 bypasses blocking

Signal Handling

Signal	Used For
SIGCHLD	Reap dead container processes via waitpid()
SIGTERM / SIGINT	Graceful supervisor shutdown — drain logs, stop containers
SIGPIPE	Ignored (SIG_IGN) — container closed pipe; write() returns EPIPE
SIGKILL	Sent by kernel monitor on hard limit breach (uncatchable)

Async-Signal-Safety Rule

Signal handlers can interrupt ANY point — inside malloc, printf, pthread_mutex_lock.

NEVER call malloc/free, printf, or FILE* I/O inside a handler.

Use the flag pattern: handler sets volatile sig_atomic_t; real work runs in the event loop.

```
volatile sig_atomic_t g_got_sigchld = 0;

static void sigchld_handler(int sig) {
  g_got_sigchld = 1; // ONLY safe op
}

// In event loop:
if (g_got_sigchld) {
  g_got_sigchld = 0;
  reap_children();
}
```

Reaping Children — waitpid Loop

while ((pid = waitpid(-1, &status, WNOHANG)) > 0) { /* classify exit */ } ← Loop required: Linux may coalesce multiple SIGCHLD signals into one delivery. Without the loop, unreaped zombies accumulate until the PID table exhausts.

Supervisor Event Loop & Thread Architecture

Main Thread

select() event loop

Waits on control socket with 1s timeout. Accepts CLI connections, dispatches handlers. Checks SIGCHLD flag. Handles shutdown.

Logger Thread

shared, single instance

bb_pop() from bounded buffer. write() log lines to <id>.log on disk. On shutdown: drain all items first.

Reader Thread

one per container

read() from container's pipe write end. bb_push() into bounded buffer. Exits when pipe returns EOF.

Container Launch Sequence (CMD_START)

1. Validate: id unused, slot available
2. Open log file: logs/<id>.log
3. pipe2(pipefd, O_CLOEXEC)
4. malloc(STACK_SIZE) — clone stack
5. `clone(child_fn, …CLONE_NEWPID|UTS|NS|SIGCHLD…)`
6. free(stack); close(pipefd[1]) in parent
7. `ioctl MONITOR_REGISTER — kernel module`
8. pthread_create — start reader thread

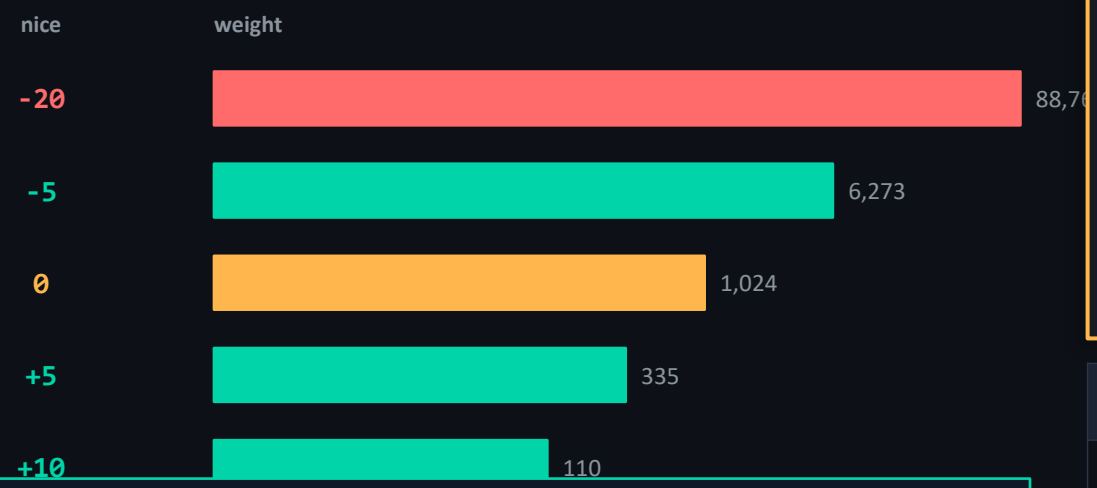
select() vs epoll()

select() scans all fds 0→nfds: O(n) per call. epoll returns only ready fds: O(1).

With ≤32 containers select() scans ≤33 fds — overhead unmeasurable. epoll adds ~150 lines of setup for zero benefit at this scale. select() is also POSIX-portable.

Linux CFS Scheduling & nice() Values

The Completely Fair Scheduler assigns CPU proportional to each task's weight, derived from its nice value.
Task with smallest vruntime (virtual runtime) runs next.



Weight ratio ≈ 1.25× per nice level

nice -5 (weight 6273) vs nice +10 (weight 110):
6273 / (6273+110) ≈ 98.3% CPU
when both are runnable → ~57× more time.

setpriority() not nice()

nice() returns -1 on BOTH success (when prev nice = -1) and on error — ambiguous.
setpriority(PRIO_PROCESS, 0, nice_value) uses errno correctly.

Experiment	Setup	Expected Result
CPU Priority	cpu_hog --nice +10 vs --nice -5	beta gets ~4–8× more CPU than alpha
I/O vs CPU	cpu_hog --nice 0 vs io_pulse --nice 0	io_pulse stays responsive (sleeps voluntarily)
Hard Limit	memory_hog 16 500 --hard-mib 64	SIGKILL when RSS > 64 MiB;

Design Tradeoffs

PID+UTS+Mount only

Network ns needs veth/bridge plumbing; User ns needs UID mapping. Three is enough to teach isolation.

mutex+cond var buffer

Lock-free ring buffers need arch-specific barriers and are notoriously hard to verify. Correctness matters more than µs latency.

spinlock in kernel

Timer callback runs in softirq — cannot sleep. mutex_lock() in softirq causes kernel panic. Spinlock is the only correct choice.

Two IPC channels

Multiplexing log data + control commands needs framing. Separation of concerns = cleaner, auditable code.

select() not epoll()

≤33 fds, O(n) cost is negligible. epoll adds 150 LOC for zero measurable gain at this scale.

Packed fixed structs

Zero serialization overhead. No external library. Padding-free across compilers. Trade: field reorder breaks compat.

Lines you must be able to explain:

- `clone(…SIGCHLD…) → enables waitpid()`
- `close(pipefd[1]) in parent → EOF detection`
- `while() cond_wait → spurious wakeup safety`
- `copy_from_user() → different page tables`
- `spin_lock not mutex in softirq → cannot sleep`
- `chroot() + chdir('/') → seal escape path`